

Applied Project — Guidelines

Design, build, and demo an end-to-end big data analytics pipeline that uses at least three (3) tools from this course — with real data flowing between them. Extra credit is available for turning the project into a publication or a valid open-source contribution.

Requirements

- **Minimum 3 tools.** Data must actually flow from one tool into the next — the tools have to be genuinely integrated.
- **Team size.** 3 students per team.
- **Problem scope.** Choose a dataset and question substantial enough that a single tool would genuinely struggle with it, either the data volume, the need for both batch and query-style access, or the need for an ML, graph, or streaming technique the other tools don't provide.

Possible Ways to Combine Three Tools

Below are five common ways to combine three or more course tools into one working pipeline. Use one as a starting point, or propose your own combination in the proposal, the only real rule is that the tools form a genuine pipeline, where each stage's output feeds the next.

A — Batch ETL & Warehousing: HDFS → Pig → Hive

Raw data lands in HDFS. Pig handles the messy transformation work - deduplication, type casting, joining multiple raw sources, filtering out bad records. Hive then sits on top as a queryable warehouse for reporting. This is the most approachable of the five combinations and suits problems that are fundamentally about structured reporting: sales dashboards, log summaries, operational metrics. It's a good choice if your team is newer to distributed tools and wants a combination where each tool's role is easy to reason about.

B — Machine Learning at Scale: HDFS → Spark → Spark MLlib

Data sits in HDFS, Spark core handles distributed loading and feature preparation, and MLlib trains and evaluates a model — classification, regression, or clustering across the full dataset rather than a sample pulled onto one machine. Suits prediction-oriented problems: churn, fraud detection, credit risk, recommendation scoring. This track leans more heavily on the analytical component of the rubric, so the model and its evaluation need to be genuinely sound, not just present.

C — Graph Analytics: HDFS → Spark → Spark GraphX

Appropriate when the data is naturally relational - a social network, a citation network, a transaction network. HDFS stores the raw vertex and edge lists, Spark loads and prepares them, and GraphX runs Pregel-style iterative algorithms (PageRank, connected components, shortest paths) to surface structure that a purely tabular tool would miss entirely. This is usually the most technically distinctive of the five tracks, since graph algorithms don't map onto the other tools at all.

D — Real-Time Pipeline: Kafka → Spark Streaming → Hive / HDFS

For problems where freshness matters more than historical depth. Kafka ingests a continuous stream of events, Spark Streaming processes it in micro-batches - aggregating, filtering, scoring - and results land in Hive or HDFS for downstream querying or a live dashboard. Suits monitoring, alerting, or anything with a “right now” requirement, such as sensor data or live activity feeds. This track carries the most infrastructure overhead of the five, so budget extra time in Week 1 to get Kafka running reliably before building on top of it.

E — Cross-Cutting: ETL + Warehouse + ML: Pig → Hive → Spark MLlib

The most complete story of the five, and the one we'd suggest if your team doesn't already have a strong topic in mind. Pig does the heavy cleaning, Hive provides a structured, query-ready layer you can use to validate the cleaning worked, and Spark MLlib runs the actual analysis on top of that clean, queryable data. It touches three genuinely different categories of tool - scripting-style ETL, a SQL-style warehouse, and distributed ML- which tends to produce both the strongest demo and the clearest tool-by-tool justification in the report.

Deliverables

- **Proposal.** Problem statement, dataset source, chosen tool combination (3+), and expected outcome.
- **Progress check-ins.** At least one interim update showing a working ingestion stage, and one showing draft analytical results, before the final submission.
- **Code + README.** The full pipeline, with setup and run instructions and a list of tools and versions used.
- **Final report.** 6–10 pages covering architecture, tool-by-tool justification, results, challenges encountered, and for teams, an individual contribution breakdown.
- **Demo.** Live walkthrough or a video no longer than 10 minutes, showing the full pipeline actually running end to end.

Evaluation - Out of 20

- **Proposal & Planning - 2 pts.** Clarity of problem statement, feasibility, tool-choice justification.
- **Implementation & Integration - 7 pts.** Correctness, real use of 3+ tools, genuine data flow between stages.
- **Analytical Quality - 3 pts.** Depth and validity of the analysis — not just that a job ran, but that the result means something.
- **Demo - 3 pts.** Clarity, evidence of real execution, honest scoping.
- **Documentation & Report - 3 pts.** Completeness, architecture diagram, reproducibility of setup instructions.
- **Presentation & Q&A - 2 pts.** Ability to explain and defend design choices.

Publication or Open-Source Contribution

On top of, not instead of, the 20 points above:

- **Publication: +3 pts.** A paper accepted to (or submitted and under review at) a peer-reviewed venue.
- **Major open-source contribution: +2 pts.** A functional pull request merged into a recognized open-source big-data project (Apache Pig, Hive, Spark, Kafka, or an established tool in that ecosystem).
- **Minor open-source contribution: +1 pt.** A documentation or small-fix pull request merged into a recognized open-source project.
- **Does not count:** a blog post or LinkedIn write-up; a public repo with no external engagement; a pull request opened but not reviewed or merged by the deadline.